705.641.81: Natural Language Processing Self-Supervised Models

Homework 6: Adapting Transformers Language Models for Your Problems

For homework deadline and collaboration policy, please see our Canvas page.

Name: Dontavious Jennings

Collaborators, if any: _____

Sources used for your homework, if any: _____

This assignment will return to basics of Transformers and dig deeper into the fundamental concepts. Additionally, we will get our hands dirty by fine-tuning Transformer language models and comparing their performance.

Homework goals: After completing this homework, you should be comfortable with:

- Training a pre-trained Language Model for your tasks
- You will explore different variations of model training
- You will compare the above results with few-shot prompting of models.

Concepts, intuitions and big picture

1. How does the BERT model expect a pair of sentences to be processed?
- `Tokens-of-sentence-1 [SEP] Tokens-of-sentence-2`
- `[CLS] Tokens-of-sentence-1 Tokens-of-sentence-2`
- `[CLS] Tokens-of-sentence-1 [SEP] Tokens-of-sentence-2`

2. When should you train a new tokenizer?
- When your dataset is similar to that used by an existing pretrained model, and you want to pretrain a new model
- When your dataset is similar to that used by an existing pretrained model, and you want to fine-tune a new model using this pretrained model
- When your dataset is different from the one used by an existing pretrained model, and you want to pretrain a new model

3. Select the sentences that apply to the BPE model of tokenization.
- BPE is a subword tokenization algorithm that starts with a small vocabulary and learns merge rules.
- BPE is a subword tokenization algorithm that starts with a big vocabulary and progressively removes tokens from it.
- BPE tokenizers learn merge rules by merging the pair of tokens that is the most frequent.
- A BPE tokenizer learns a merge rule by merging the pair of tokens that maximizes a score that privileges frequent pairs with less frequent individual parts.
- BPE tokenizes words into subwords by splitting them into characters and then applying the merge rules.
- BPE tokenizes words into subwords by finding the longest subword starting from the beginning that is in the vocabulary, then repeating the process for the rest of the text.

4. What are the labels in a masked language modeling problem?
- Some of the tokens in the input sentence are randomly masked and the labels are the original input tokens.
- Some of the tokens in the input sentence are randomly masked and the labels are the original input tokens, shifted to the left.
- Some of the tokens in the input sentence are randomly masked, and the label is whether the sentence is positive or negative.
- Some of the tokens in the two input sentences are randomly masked, and the label is whether the two sentences are similar or not.

5. When should you pretrain a new model?

- When there is no pretrained model available for your specific language
- When you have lots of data available, even if there is a pretrained model that could work on it
- When you have concerns about the bias of the pretrained model you are using

. Is there something wrong with the following code?

```
from transformers import AutoTokenizer, AutoModel

tokenizer = AutoTokenizer.from_pretrained("bert-base-cased") model = AutoModel.from_pretrained("gpt2")

encoded = tokenizer("Hey!", return_tensors="pt") result = model(**encoded)
```

- No, it seems correct.
- The tokenizer and model should always be from the same checkpoint.
- It's good practice to pad and truncate with the tokenizer as every input is a batch.

1. [CLS] Tokens-of-sentence-1 [SEP] Tokens-of-sentence-2

2. When your dataset is different from the one used by an existing pretrained model, and you want to pretrain a new model.

3. BPE is a subword tokenization algorithm that starts with a small vocabulary and learns merge rules; BPE tokenizers learn merge rules by merging the pair of tokens that is the most frequent; and BPE tokenizes words into subwords by splitting them into characters and then applying the merge rules.

4. Some of the tokens in the input sentence are randomly masked and the labels are the original input tokens.

5. When there is no pretrained model available for your specific language, and when you have concerns about the bias of the pretrained model you are using.

6. The tokenizer and model should always be from the same checkpoint.

Getting Your Attention with Self-Attention

Recall that the transformer architecture uses scaled dot-product attention to compute *attention weights*:

$$\boldsymbol{\alpha}^{(t)} = \text{softmax}\left(\frac{\mathbf{q}_t \mathbf{K}^\top}{\sqrt{h}}\right) \in [0,1]^n$$

The resulting embedding in the output of attention at position t are:

$$\text{Attention(Q,K,V)}_t = \sum_{t\prime=1}^{n} \alpha_{t\prime}^{(t)} \, \text{v}_{t\prime} \in \mathbb{R}^{1\times h},$$

where $\boldsymbol{\alpha}^{(t)} = \left[\alpha_0^{(t)}, \ldots, \alpha_n^{(t)}\right]$. The same idea can be stated in a matrix form,

$$\text{Attention(Q,K,V)} = \text{softmax}\left(\frac{\text{QK}^\top}{\sqrt{h}}\right)\text{V} \in \mathbb{R}^{n\times h}.$$

In the above equations:

- h is the hidden state dimension and n is the input sequence length;
- $\text{X} \in \mathbb{R}^{n\times h}$ is the input to the attention;
- $\text{x}_t \in \mathbb{R}^{1\times h}$ is the slice of X at position t, i.e. vector representation (embedding) of the input token at position t;
- $\text{W}_q, \text{W}_k, \text{W}_v \in \mathbb{R}^{h\times h}$ are the projection matrices to build query, key and value representations;
- $\text{Q} = \text{XW}_q \in \mathbb{R}^{n\times h}$,$\text{K} = \text{XW}_k \in \mathbb{R}^{n\times h}$,$\text{V} = \text{XW}_v \in \mathbb{R}^{n\times h}$ are the query, key and value representations;
- $\text{q}_t = \text{x}_t\text{W}_q \in \mathbb{R}^{1\times h}$ is the slice of Q at position t. Similarly, $\text{k}_t = \text{x}_t\text{W}_k \in \mathbb{R}^{1\times h}$ and $\text{v}_t = \text{x}_t\text{W}_v \in \mathbb{R}^{1\times h}$.

Forget 'Softmax', 'Argmax' is the New Boss in Town!

Recall from lectures that we can think about attention as being queryable softmax pooling. In this problem, we ask you to consider a hypothetical 'argmax' version of attention where it returns exactly the value corresponding to the key that is most similar to the query, where similarity is measured using the traditional inner-product. In other words, here use a version of Attention that instead of using softmax we use argmax to generate outputs from the attention layer. For example, `softmax([1, 3, 2])` becomes `argmax([1, 3, 2]) = [0, 1, 0]`.

1. Perform argmax attention with the following keys and values:

$$\text{Keys} = \left\{\begin{bmatrix}0\\1\\2\end{bmatrix}, \begin{bmatrix}1\\1\\2\end{bmatrix}, \begin{bmatrix}-1\\1\\-2\end{bmatrix}, \begin{bmatrix}0\\-1\\-2\end{bmatrix}\right\}, \text{Values} = \left\{\begin{bmatrix}3\\2\\-4\end{bmatrix}, \begin{bmatrix}3\\-2\\4\end{bmatrix}, \begin{bmatrix}3\\2\\0\end{bmatrix}, \begin{bmatrix}3\\2\\4\end{bmatrix}\right\}$$

   using the following query:

$$q = \begin{bmatrix} -2 \\ 1 \\ 1 \end{bmatrix}$$

What would be the output of the attention layer for this query? Remember, to simplify calculations, use an argmax instead of softmax. Note about notation: each vector in 'Keys = {.}' and 'Values{.}' are key/value vectors corresponding to four, non-interchangeable positions (i.e., ordering of these vectors matter).

[3, 2, -4]. The dot products of q = [-2, 1, 1] with the four keys are 0 + 1 + 2 = 3, -2 + 1 + 2 = 1, 2 + 1 - 2 = 1, and 0 - 1 - 2 = -3. The first key has the largest score, so argmax gives the weights [1, 0, 0, 0] and the output is the first value vector, [3, 2, -4].

2. How does this design choice affect our ability to usefully train models involving attention? (**Hint:** think about how the gradients flow through the network in the backward pass. Can we learn to improve our queries or keys during the training process?)

It means the attention weights can't be learned with gradient descent. Argmax is piecewise constant, so a small change to a query or key doesn't change which position wins, and the gradient of the output with respect to the queries and keys is 0 almost everywhere (and undefined where two scores tie). The selected value vector still gets a gradient, so W_v can be trained, but W_q and W_k never get a signal, so the model can't learn where to attend.

Superposition of Information in Self-Attention

We will show that the attention mechanism is able to copy the information from its input, whenever needed.[1]

1. **'Copying' mechanism in self-attention:** We'll first show that it is particularly simple for attention to "copy" a value vector to the output. Describe (in 1-2 sentences) what properties of the inputs to the attention operation would result in the output $\text{Attention}(Q,K,V)_t$ being approximately equal to v_j for some $j \in [0 \dots n]$. **Hint:** Show that there can exist $j \in [0 \dots n]$ such that $\alpha_j^{(t)} \gg \sum_{i \neq j} \alpha_k^{(t)}$, if certain property holds for the query q_t, and the keys $\{k_0 \dots k_n\}$.

If the query q_t has a much larger dot product with one key k_j than with every other key, for example when q_t points in the same direction as k_j with a large norm and is close to

[1] Question credit: John Hewitt

orthogonal to the other keys, the softmax puts almost all of its weight on position j. Then α_j ≫ Σ_(i≠j) α_i, and the output is approximately v_j.

2. **Extracting signals after averaging them:** Instead of focusing on just one vector v_j, attention mechanism might want to incorporate information from multiple source vectors. Consider the case where we instead want to incorporate information from two vectors v_a and v_b, with corresponding key vectors k_a and k_b. How should we combine information from two value vectors v_a and v_b? A common way to combine values vectors is to average them: $\bar{v} = \frac{1}{2}(v_a + v_b)$. However, after such averaging it is not quite clear how to tease apart the original information in value vectors v_a and v_b. Unless … some special properties hold!
   Suppose that although we don't know v_a or v_b, we do know that v_a lies in a subspace[2] A formed by the m basis vectors[3] $\{a_1, a_2, \ldots, a_m\}$, while v_b lies in a subspace B formed by the p basis vectors $\{b_1, b_2, \ldots, b_p\}$. This means that any v_a can be expressed as a linear combination of its basis vectors, as can v_b. All basis vectors have norm 1 and are orthogonal to each other. Additionally, suppose that the two subspaces are orthogonal; i.e. $a_j^\top b_k = 0$ for all j, k. Using the basis vectors $\{a_1, a_2, \ldots, a_m\}$, construct a matrix M such that for arbitrary vectors $v_a \in A$ and $v_b \in B$, we can use M to extract v_a from the average vector $\bar{v}$. In other words, we want to construct M such that for any v_a, v_b, $M\bar{v} = v_a$. Show that $M\bar{v} = v_a$ holds for your M.

Let A = [a_1, a_2, …, a_m] be the matrix whose columns are the basis vectors of subspace A, and set M = 2 A A^T = 2 Σ_(i=1..m) a_i a_i^T.

Check: since v_a is in A, it can be written as v_a = Σ_j c_j a_j. Then A A^T v_a = Σ_i a_i (a_i^T Σ_j c_j a_j) = Σ_i c_i a_i = v_a, because a_i^T a_j is 1 when i = j and 0 otherwise. Every a_i is orthogonal to every b_k, so A^T v_b = 0 and A A^T v_b = 0.

So M v̄ = 2 A A^T (½ (v_a + v_b)) = A A^T v_a + A A^T v_b = v_a + 0 = v_a. The factor of 2 undoes the ½ from the average; without it, A A^T v̄ would only give ½ v_a.

3. **Averaging pairs of representations:** As before, let v_a and v_b be two value vectors corresponding to key vectors k_a and k_b, respectively. Assume that (1) all key vectors are orthogonal, so $k_i^\top k_j = 0$ for all $i \neq j$; and (2) all key vectors have norm 1. Find an expression for a query vector q_t such that $\text{Attention}(Q,K,V)_t = \frac{1}{2}(v_a + v_b)$ and justify your answer.
   More here: https://web.stanford.edu/class/cs224n/assignments/a5.pdf

---

[2] https://en.wikipedia.org/wiki/Linear_subspace

[3] https://en.wikipedia.org/wiki/Basis_(linear_algebra)

Take q_t = β (k_a + k_b), where β is a large positive number. Since the keys are orthonormal, positions a and b both get a score of β/√h, while the other positions get 0. As β increases, the attention weights for a and b approach 0.5 each and the others approach 0, so the output approaches (v_a + v_b)/2. With a finite β and additional keys, this is an approximation rather than an exact equality.

Extra Credit: Importance of 'Scaling' in Self-Attention

In practice, we scale each dot product QK^T by a factor of $\sqrt{h}$. This is called *scaled dot product attention*. In this part, we will prove why we perform this scaling. Suppose we are performing a dot product between a key k and query q, where $k,q \in \mathbb{R}^{1\times h}$ and $k,q \sim \mathcal{N}(\boldsymbol{\mu}, \sigma^2\mathbf{I})$. You can assume that k,q are sampled independently.

1. Compute $\mathbf{E}\left[qk^T\right]$ in terms of $\boldsymbol{\mu}, h, \sigma$.

E[q k^T] = h μ^2. q k^T = Σ_(i=1..h) q_i k_i, and since q and k are independent, E[q_i k_i] = E[q_i] E[k_i] = μ · μ, so the sum over h dimensions is h μ^2.

2. Compute $\mathrm{Var}\left[qk^T\right]$ in terms of $\boldsymbol{\mu}, h, \sigma$.

Var[q k^T] = h (σ^4 + 2 μ^2 σ^2). The h terms q_i k_i are independent, so their variances add. For one term, Var(q_i k_i) = E[q_i^2] E[k_i^2] - (E[q_i] E[k_i])^2 = (σ^2 + μ^2)^2 - μ^4 = σ^4 + 2 μ^2 σ^2. Multiplying by h gives the result.

3. Based on the variance computed above and assuming that $\boldsymbol{\mu} = 0$ and $\sigma = 1$, explain why we need to scale the dot product by $\sqrt{h}$. Explain why this scaling is important for numerical stability.

With μ = 0 and σ = 1, the dot product has variance h and standard deviation √h. At h = 768, the standard deviation is about 28, which can make softmax heavily favor one position and reduce the gradient signal to the others. Dividing by √h brings the variance back to 1 and keeps the scores in a more useful range. It also reduces the risk of overflow when softmax is computed without numerical stabilization.

Programming

In this programming homework, we will finetune encoder LMs for a classification task with yes/no questions. In particular, we will

- Implement full-parameter finetuning.
- Explore various parameter-efficient finetuning strategies.

Skeleton Code and Structure:

The code base for this homework can be found at this GitHub repo under the `hw6` directory. Your task is to fill in the missing parts in the skeleton code, following the requirements, guidance, and tips provided in this pdf and the comments in the corresponding .py files. The code base has the following structure:

- `base_classification.py`: implements the basic full finetuning
- `classification.py`: implements different parameter-efficient finetuning

TODOs — Your tasks include 1) generate plots and/or write short answers based on the results of running the code; 2) fill in the blanks in the skeleton to complete the code. We will explicitly mark these plotting, written answer, and filling-in-the-blank tasks as **TODOs** in the following descriptions, as well as a **`# TODO`** at the corresponding blank in the code.

Submission:

Your submission should contain two parts: 1) plots and short answers under the corresponding questions below; and 2) your completion of the skeleton code base, in a `.zip` file, which includes the two provided skeleton files and a `openai_gpt-boolq.py` script you created (details in 3.3).

Overview and Dataset

In this programming assignment, you will get your hands dirtier with building self-supervised models. You will build a classifier using the Huggingface Transformer library and PyTorch. This classifier is expected to solve the BoolQ dataset . You can find this dataset on Huggingface's dataset hub: https://huggingface.co/datasets/boolq. In this task, each input consists of a question statement and a passage that may contain the answer to this question. The output is the answer to the question which may be one of 'Yes' or 'No' labels. Here is an example:

"passage": "Äll biomass goes through at least some of these steps: it needs to be grown, collected, dried, fermented, distilled, and burned...", "question": "does ethanol take more energy make that produces" "answer": false,

The data comes with training and validation subsets. We will select a subset of the training data as 'test' set. So for the main experiments we will use 8k instances for training and the rest for testing.

Python Scripts Arguments

Many of the **TODOs** below involves setting different training configurations and hyper-parameters, this could be achieved through the arguments provided in `main` of both `base_classification.py` and `classification.py` - they should be self-explanatory.

General Hint

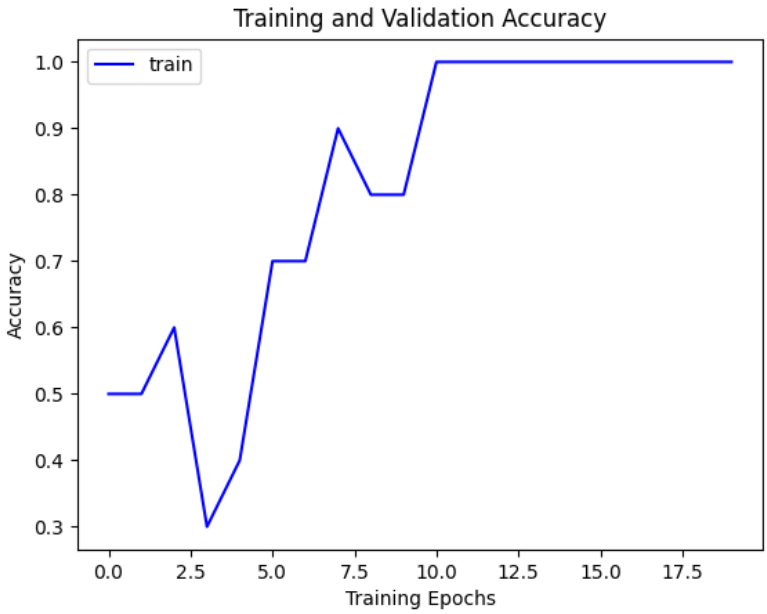
Follow the specifications and requirements in the code comments to complete missing lines.

Finetuning: Basics

1. Let's start with the basic full finetuning, where all the parameters will be updated during the training. Specifically, we will use `AutoModelForSequenceClassification` which implements a classifier that maps the representation of encoder LMs (BERT, RoBERTa, etc.) into a fixed set of labels. Use this class to build a binary classifier using the representations of `distilbert-base-uncased` . The starter code for full finetuning in `base_classification.py` has a bunch of basic functionalities implemented, but it is missing details.
   TODOs: Read and complete the missing lines in `evaluate_model` and `train` functions in `base_classification.py` to complete the functionalities of full finetuning.

   To make sure your implementation is correct, we can train the model on 10 training instances for a few epochs (say, a total of 20 epochs). Note that over-fitting a few training examples is a good first test to make sure your implementation can successfully learn the patterns of your data. You should be able to complete the starter code and run it on a few instances in your local machine. You should be able to see that your classifier successfully over-fits its few training examples.
   TODOs: Run `base_classification.py` with the following command on your local machine, paste the generated plot of training accuracy vs. epochs plots below.

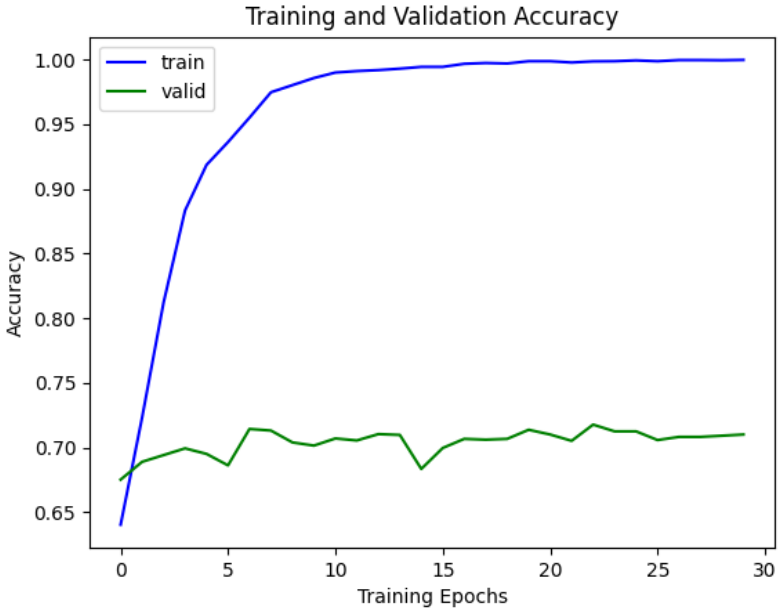
   > python base_classification.py –small_subset –device cuda –model "distilbert-base-uncased" –batch_size "64" –lr 1e-4 –num_epochs 20



Training accuracy on the 10 examples started at 50%, moved around for the first few epochs, and reached 100% at epoch 11. It stayed at 100% for the rest of the 20 epochs, so the model was able to memorize the small training set, which suggests the training loop is working.

2. Now, rather than running training on a subset of the data, try training with all the data on your compute environment.

   TODOs Train for 30 epochs and paste the generated plot of train and dev accuracies vs. epochs below. Also report the largest batch size.
   Hint: You should see that training accuracy goes high, while the dev accuracy will go high and then turn downhill as the model overfits the data.



I trained DistilBERT on all 8,000 training examples for 30 epochs with a batch size of 64 and a learning rate of 1e-4. Training accuracy went from 64.0% after the first epoch to 99.96% by the end. Dev accuracy improved from 67.5% to about 70% in the first few epochs, peaked at 71.8% in epoch 23, and otherwise moved between about 68% and 72%. Instead of dropping sharply, it flattened out while training accuracy kept going up. That is still overfitting, since the model kept fitting the training set without getting better on new data. The final dev accuracy was 71.0% and the test accuracy was 72.0%.

Largest batch size: 544. I found it by doubling the batch size until training ran out of memory and then narrowing the range, using DistilBERT with the same max length of 128 on my 24 GB GPU. A batch of 560 ran out of memory.

3. Let's implement hyper-parameter selection by exploring the combination of different hyper-parameter choices.

   TODOs: Train models for various choices of learning rates (`1e-4, 5e-4, 1e-3`) and training epochs (`7, 9`). Select the model with the highest accuracy on the dev set and report its accuracy on the test set. Also report the corresponding choice of hyperparameters

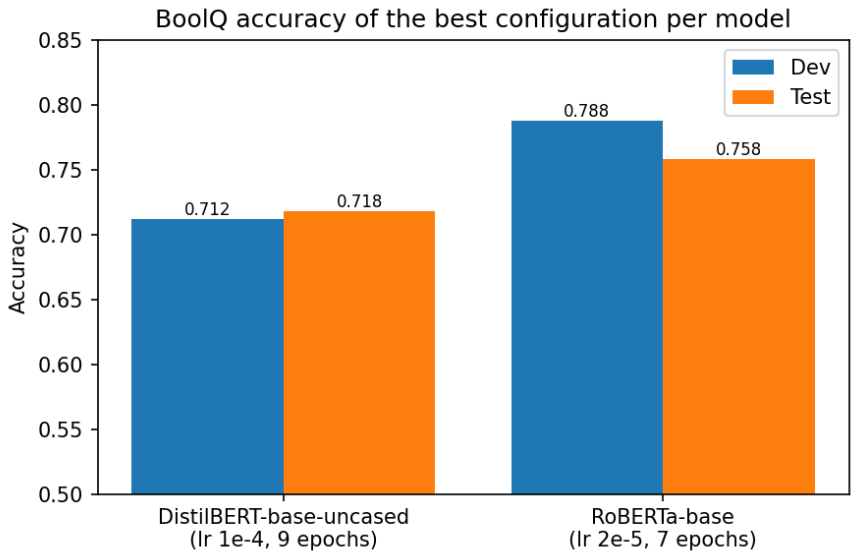
Learning rate	Epochs	Dev accuracy	Test accuracy

1e-4	7	71.0%	71.6%
1e-4	9	71.2%	71.8%
5e-4	7	62.2%	62.3%
5e-4	9	62.2%	62.4%
1e-3	7	62.2%	60.5%
1e-3	9	62.2%	61.3%

The best model on the dev set used a learning rate of 1e-4 and 9 epochs, with 71.2% dev accuracy and 71.8% test accuracy. All runs used DistilBERT with a batch size of 64.

Both of the larger learning rates failed. The model ended up predicting "yes" for every question, which gives 62.2% on the dev set because that is the share of "yes" answers there, and training accuracy for those runs also stayed around 62%. The test numbers for those runs differ a little because base_classification.py shuffles the data without a seed, so each run gets a slightly different test split.

4. **TODOs** Now repeat the previous experiment with other models. In particular, create a bar plot that shows test/dev accuracies for two of your favorite of the following models: `distilBERT-base-uncased`, `BERT-base-uncased`, `BERT-large-uncased`, `BERT-base-cased`, `BERT-large-cased`, `RoBERTa-base`, `RoBERTa-large`. Note, it is possible that some of these models wouldn't fit in your compute environment for any choice of batch size, in which case report 0 for their performance.



I compared DistilBERT-base-uncased with RoBERTa-base, using a batch size of 64 and picking each model's best run by dev accuracy. DistilBERT's best run was the one from

3.1.3 (learning rate 1e-4, 9 epochs), with 71.2% dev and 71.8% test accuracy. RoBERTa-base's best run used a learning rate of 2e-5 and 7 epochs, with 78.8% dev and 75.8% test accuracy.

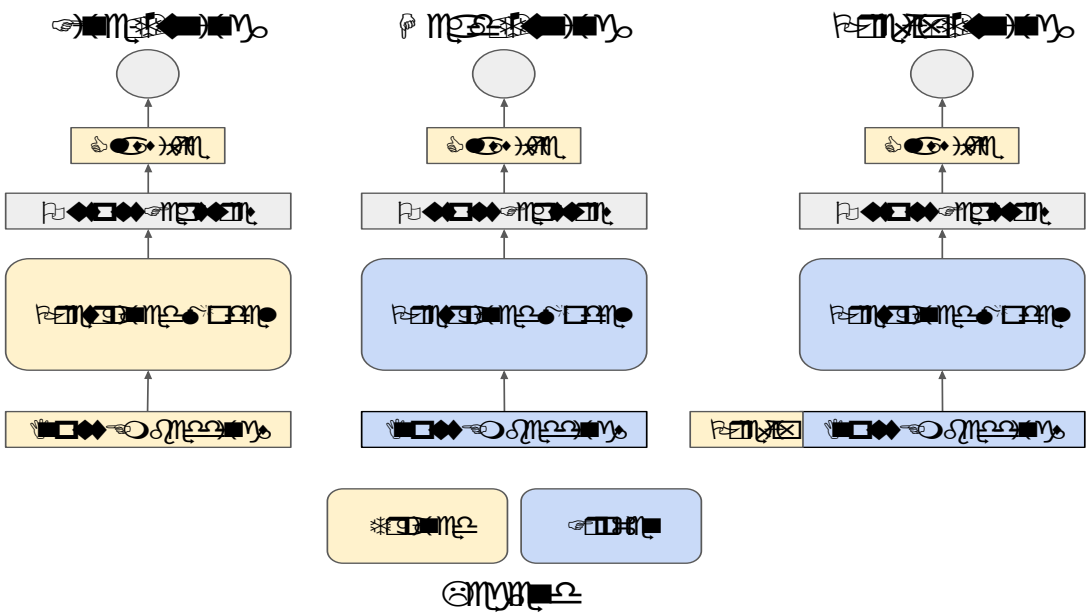
With the same grid as 3.1.3, RoBERTa-base collapsed to always predicting "yes" in all six runs (62.2% dev), even at 1e-4. RoBERTa usually needs a smaller learning rate, so I added runs at 1e-5 and 2e-5, and all four of those trained normally:

Learning rate	Epochs	Dev accuracy	Test accuracy
1e-5	7	76.4%	77.6%
1e-5	9	78.0%	74.7%
2e-5	7	78.8%	75.8%
2e-5	9	78.5%	78.6%
1e-4, 5e-4, 1e-3	7 and 9	62.2% (always "yes")	60.6% to 63.9%

RoBERTa-base performed about 8 percentage points better than DistilBERT on the dev set. It has 12 layers compared with DistilBERT's 6 and was pretrained on more data. Those differences may help explain the results, but this experiment does not isolate which one mattered.

Parameter-Efficient Fine-Tuning

In this section, we will implement two more efficient ways of tuning language models: **Head-Tuning**, which freezes all parameters but the classification head, and **Prefix-Tuning**, which also freezes all parameters but the classification head, and additionally appends a fixed length ℓ of trainable embeddings to the input embeddings. See figure 1 for an illustration of which parameters are getting updated in each fine-tuning method we will explore in this homework.
Note that “output features" tensor for a single sequence of input should be of shape (length,hidden size), or equivalently $\mathbb{R}^{N\times d}$. However, the classifier expects a tensor of shape $\mathbb{R}^{1\times d}$. There are multiple ways to transform the output features to $\mathbb{R}^{1\times d}$, but in this work, we will take the **averaged** token representation.
Instead of using `AutoModelForSequenceClassification`, we now use the base `AutoModel` class to tailor the model to our needs. But don’t worry, a skeleton code is provided to you in `classification.py`, you would only need to modify the details.



Overview of fine-tuning methods. Blue indicates parts that are frozen (no gradient update) during the fine-tuning process. Yellow indicates parts that are trained (with gradient updates).

1. For comparison, let's redo the full finetuning first. Change the base model to `RoBERTa-base`. Implement full fine-tuning by adding a binary classifier on the averaged sentence representation.

   TODOs: Read and complete the missing lines

   a. in `evaluate_model` function

   b. after `for index, batch in tqdm(enumerate(train_dataloader)):` in `train` function

   in `classification.py` to complete the functionalities of full finetuning.
   Hint: Your implementation should be the same as that of `base_classification.py` for `evaluate_model` and almost the same except for how we get the logtis for `train` (until loss backward).
   TODOs: Read and complete the missing lines after `if self.type == 'full'` in the `forward` function of the `CustomModelforSequenceClassification` class in `classification.py`. Run the code with `type="full"` to train the model with full finetuning. Report the final accuracy you obtained along with the hyper-parameters you used (number of epochs and learning rate).

Final accuracy: 75.6% on the dev set and 76.5% on the test set.

Hyperparameters: learning rate 1e-5, 5 epochs, batch size 32, with the AdamW optimizer and linear schedule from the starter code. I used 1e-5 because RoBERTa-base collapsed to always predicting "yes" at every learning rate in the 3.1 grid. Dev accuracy went up every epoch, from 67.3% to 75.6%, and was still slowly rising at epoch 5.

2. **TODOs**: Write down the number of tuned parameters for head-tuning for BoolQ, given that the model hidden size is d.

   Now let's implement head-tuning by adding a binary classifier on the averaged sentence representation.
   TODOs: Read and complete the missing lines after `if self.type == 'head'` in the `forward` function of the `CustomModelforSequenceClassification` class in `classification.py`. You would also need to modify the optimizer by adding code after `mymodel.type == "head":` in the `train` function in `classification.py`. Report the final accuracy on full head-tuning along with the hyper-parameters you used[4].

2d + 2. Head-tuning only trains the classifier, a linear layer from the averaged d-dimensional representation to 2 labels, which has 2d weights and 2 biases. For RoBERTa-base (d = 768) that is 1,538 parameters.

Final accuracy: 64.2% on the dev set and 65.1% on the test set, with a learning rate of 1e-3, 10 epochs, and a batch size of 32.

This is only about 2 percentage points above the 62.2% always-yes baseline. With the encoder frozen, the classifier only learns from RoBERTa's averaged representation, and that setup did not improve much on this task. Each epoch took about 18 seconds compared with 32 seconds for full fine-tuning, since no gradients are computed through the encoder.

3. **TODOs**: Write down the number of tuned parameters for our prefix-tuning on BoolQ that appends trainable prefix embeddings of length 128, given the model hidden size is d.

   It's time to implement prefix tunning!
   TODOs: Read and complete the missing lines after `if self.type == 'prefix'` in the `forward` function of the `CustomModelforSequenceClassification` class in `classification.py`. You would also need to modify the optimizer by adding code after `mymodel.type == "prefix":` in the `train` function in `classification.py`. Report the final accuracy of prefix-tuning along with the hyper-parameters you used

[4] You would be only graded by the correctness of your implementation, but if you want good performance, Try using a larger learning rate and longer training epochs.

[5].

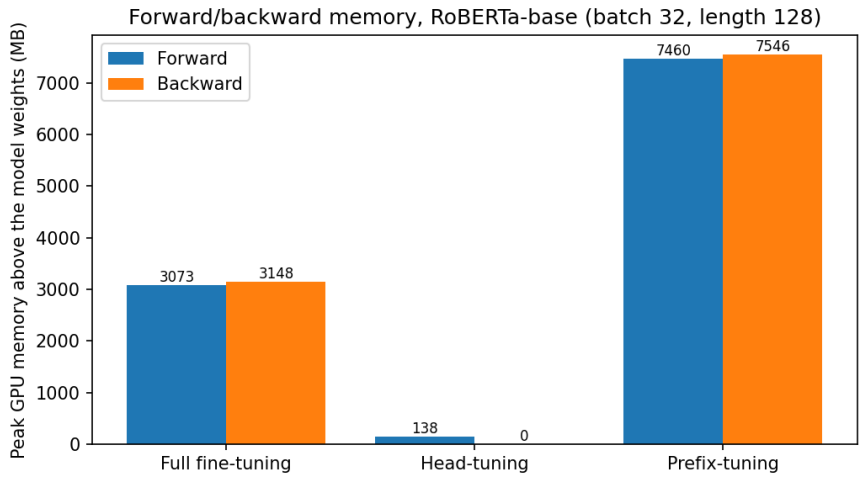
128d + 2d + 2 = 130d + 2. Prefix-tuning trains the 128 × d prefix embeddings plus the same classifier as head-tuning. For d = 768 that is 98,304 + 1,538 = 99,842 parameters.

Final accuracy: 64.4% on the dev set and 65.2% on the test set, with a learning rate of 5e-3, 10 epochs, a batch size of 32, and a prefix length of 128.

It did slightly better than head-tuning but was still far below full fine-tuning. It was also the slowest method, at about 57 seconds per epoch, because the prefix doubles the sequence length from 128 to 256 and the gradients still have to pass through the whole encoder to reach the prefix.

4. **Extra Credit:** Here is a simple tutorial on how to use torch.profile to record the memory usage of forward and backward operations.

   TODOs Generate a bar plot of the forward/backward memory usage of full-finetuning, head-tuning, and prefix-tuning. Note that you don't need to upload any code for this.



I measured the peak GPU memory above the model weights for one batch of 32. Full fine-tuning used about 3.1 GB in both passes, since every layer's activations are kept for the backward pass. Head-tuning used only 138 MB forward and almost nothing backward, because the encoder runs without gradients. Prefix-tuning used the most, about 7.5 GB,

[5] Again, you would be only graded by the correctness of your implementation, but if you want good performance, Try using a larger learning rate and longer training epochs.

because the prefix doubles the sequence length to 256 and gradients still go through the whole encoder.

5. **LoRA** fine-tuning decomposes the parameter updates of a large dense matrix to two low-rank matrices. For a matrix M of size $\mathbb{R}^{d\times d}$, the full updates require storing the gradients and optimizer states of $d \times d$ parameters.
   TODOs: However, if we decompose the gradient update on the full matrix $\Delta M \approx AB$ where $A \in \mathbb{R}^{d\times n}$ and $B \in \mathbb{R}^{n\times d}$, write down the number of parameters whose gradients and optimizer states we need to store:
   .

2dn. A has d × n entries and B has n × d entries, so we only store gradients and optimizer states for 2dn parameters instead of d^2. With n much smaller than d this is a big saving: for d = 768 and n = 8, it is 12,288 instead of 589,824.

6. **Extra Credit:** Implement **LoRA** fine-tuning.
   TODOs: Create `classification_lora.py` and copy `classification_base.py` to it and implement the LoRA fine-tuning method using the peft library. This should be just changing a few lines of code. Report the trainable parameters and final accuracy of your LoRA fine-tuning, as well as your hyper-parameters.

I created classification_lora.py by copying base_classification.py and wrapping the model with peft's LoRA (rank 8, alpha 16, dropout 0.1), which adds low-rank matrices to the attention query and value layers of RoBERTa-base.

Trainable parameters: 887,042 out of 125,534,212 (0.71%), including the classification head.

Hyperparameters: learning rate 5e-4, 5 epochs, batch size 32.

Final accuracy: 76.0% on the dev set and 75.3% on the test set. That is about the same as full fine-tuning (75.6% dev, 76.5% test) while training less than 1% of the parameters.

In-Context Learning

1. Here we will prompt language models.[6] You should sign up for the OpenAI API, which lets you use GPT-3 a large, neural language model like the ones that we learned about in lecture.

   The OpenAI API is a paid service. OpenAI will give you a few dollars in credit when you first create your account (See the usage page). For this assignment, the cost should be less than that. For the first part of the assignment, we'll get warmed up by

[6] Question credit: Chris Callison-Burch's AI course at UPenn.

playing with the OpenAI API via its interactive Playground website. Later we'll see how to integrate it directly into our code.

OpenAI

OpenAI products offer a wide spectrum of capabilities and modes of interaction with their models. For this homework, **we use the `Completion` mode for both playground and API calls**. In the future, you can explore other modes that might be more up-to-date and powerful, e.g. `Chat`(Playground, APIs), `Assistants` (Playground, APIs), and many more!

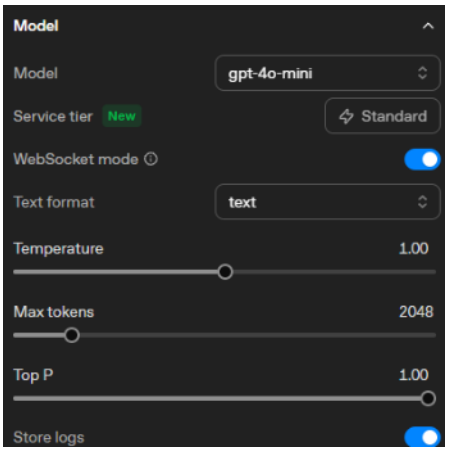
Note: we suggest you to prioritize your credit usage on finishing this homework question with `Completion` before exploring other modes, as querying powerful models might be more expensive.

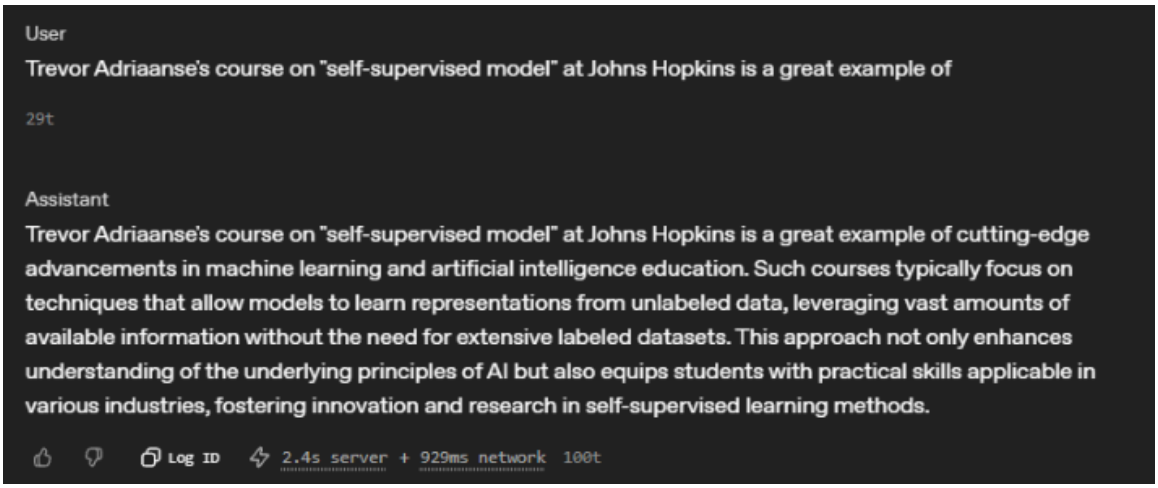
First, let's learn some basic terminology:

- Prompt: the input to the model
- Completion: what the model outputs

Let's try it out.

TODOs: Paste this prompt into the playground, press the "Generate" button, and see what it says. Paste a screenshot of the completion below:





The Completions playground and gpt-3.5-turbo-instruct have been retired, so I ran this prompt in the current Playground with gpt-4o-mini at the default settings (temperature 1).

Now save its output for the end of the semester for your course reviews … just kidding!!

2. There are several controls on the right hand side of the playground. These are:
    - **Model:** GPT models come in several different sizes and capabilities. You can read more documentation on these models here. For our homework, we use the `gpt-3.5-turbo-instruct` model.
    - **Temperature and Top P sampling:** control how the model samples tokens from its distribution. Setting Temperature to 0 will cause the model to produce the highest probability output. Setting it closer to 1 will increase its propensity to create more diverse output.
    - **Maximum length:** what's the maximum length (in tokens) that the model will output?
    - **Stop sequences:** you can specify what tokens should cause the model to stop generating. You can use the newline character or any special sequence that you designate.
    - **Frequency Penalty and Presence Penalty:** two parameters that help to limit how much repetition there is in the model's output.
    - **Best of:** Best of n indicates the model generate n completion candidates on the server side, and display the top-1 on the client (user) side.

For the remaining parts of this section, set the engine to `davinci` so that it uses vanilla language model (no additional tuning with human feedback).
In-context learning: In addition to writing awesome reviews of your professors, you can design prompts to get GPT-3 to do all sorts of surprising things. For instance,

GPT-3 can perform few-shot learning. Given a few examples of a task, it can learn a pattern very quickly and then be used for classification tasks. It often helps to tell the model what you want it to do.

Here's an example from the paper that introduced GPT-3. It shows a few-show learning example for correcting grammatically incorrect English sentences.

TODOs: Paste this prompt into the playground, press the "Generate" button, and see what it says. Paste the completion below:

Completion: Good English output: I would be more than happy to work with you on another project.

davinci has also been retired, so I used gpt-4o-mini through the API at temperature 1. It picked up the pattern from the examples and changed "in another project" to "on another project."

3. **Instruction following:** In addition to few shot learning, GPT and other large language models do a pretty remarkable job in "zero-shot" scenarios. You can give them instructions in natural language and they may produce remarkably good output.

   For example, let's try the following prompt:

   TODOs: Paste this prompt into the playground, press the "Generate" button, and see what it says. Paste the completion below:

Completion: Today I went to the store to buy some bottles of water.

With no examples, it still fixed every error, including "have went" and "some many bottle." I used gpt-4o-mini through the API at temperature 1.

4. **Programmatic access:** In addition to prompting GPT models in the playground, you can write Python code to query models with provided OpenAI APIs. Now let's use few-shot prompting to solve a few instance of BoolQ.
   TODOs: Write a Python script for few-shot prompting OpenAI GPT models on BoolQ. Specifically, form a prompt by randomly selecting 8 demonstrations from BoolQ. Make sure to have a balanced prompt (50% 'yes's and the rest 'no's) and interleave them to prevent the recency and label-imbalance biases. Using this in-context demonstration, evaluate `gpt-3.5-turbo-instruct` on a small subset of BoolQ instances (say, 30 examples). Hand in your code used to perform this evaluation, it should be named as `openai_gpt-boolq.py`. Write down the evaluation

accuracy below.
Hints:

- We use the `gpt-3.5-turbo-instruct` model (`model="gpt-3.5-turbo-instruct"`)
- You can use the playground to create a skeleton code to start with based on a prompt that you can then use in your Python projects. Click on the "View Code" button on the top-right, and you'll get some code that you can convert into a script.
- You can also refer to the full documentation for creating completion and https://platform.openai.com/docs/api-reference/completions/object.
- Using OpenAI API requests an API key. You can generate a key associated with your account here. To input the key, you can either do `export OPENAI_API_KEY=<your key>` on your compute environment before running the script. Or refer to the documentation.

Evaluation accuracy: 83.3% (25 out of 30).

openai_gpt-boolq.py picks 8 random demonstrations from the BoolQ training set, 4 with the answer yes and 4 with no, and interleaves them so the labels alternate. It then asks the model 30 random validation questions (20 yes and 10 no) at temperature 0. gpt-3.5-turbo-instruct has been retired, so the script uses gpt-4o-mini through the chat endpoint with the same few-shot prompt. Four of the five mistakes were questions where the correct answer was yes and the model said no.

Optional Feedback

Have feedback for this assignment? Found something confusing? We'd love to hear from you!